CS 595 Lab Notebook

Lab 3

Name: Jarren Calizo
OdinID: calizo
Date: April 26, 2025

Table of Contents

1. Lab Overview 2
2. Lab 3.1 Command and SQL injection
    - Step 1: OS Command Injection (Simple) 3
    - Step 2: Blind OS Command Injection with Time Delays 5
    - Step 3: Blind OS Command Injection with Output Redirection 6
    - Step 4: Blind OS Command Injection with Out-of-Band Interaction 9
    - Step 5: SQL Injection to Retrieve Hidden Data 10
    - Step 6: SQL Injection (Login Bypass) 11
    - Step 7: SQL Injection (Union Attack - Determining Number of Columns) 12
    - Step 8: SQL Injection (Union Attack – Finding a Column Containing Text) 13
    - Step 9: SQL Injection (Union Attack – Retrieving Data from Other Tables) 15
    - Step 10: SQL Injection (Union Attack – Querying the Database Version) 16
    - Step 11: SQL Injection (Examining the Database - Listing Contents) 17

Lab Overview

This lab sequence expanded my practical skills across both command injection and SQL injection exploitation, systematically demonstrating how unsanitized input can be leveraged to gain unauthorized access, manipulate backend servers, and extract sensitive database information.

The first four steps focused on OS command injection, gradually increasing in complexity. Starting with simple injections through crafted POST requests, I confirmed vulnerabilities by exfiltrating command outputs (whoami results) and manipulating server behaviors through induced delays and output redirection. As direct output was restricted, I moved toward blind techniques using time-based delays and out-of-band (OOB) network interactions to complete the challenges. These steps solidified how attackers can exploit even blind vulnerabilities by carefully observing indirect server behaviors.

Transitioning into SQL injection, I systematically tested and exploited vulnerabilities across multiple dimensions — from basic retrieval of hidden data, to bypassing authentication mechanisms through crafted login payloads. I progressed into more advanced union-based SQL injection attacks, beginning with determining the number of columns in vulnerable queries, identifying which columns accept text data, and crafting union payloads that returned sensitive table contents.

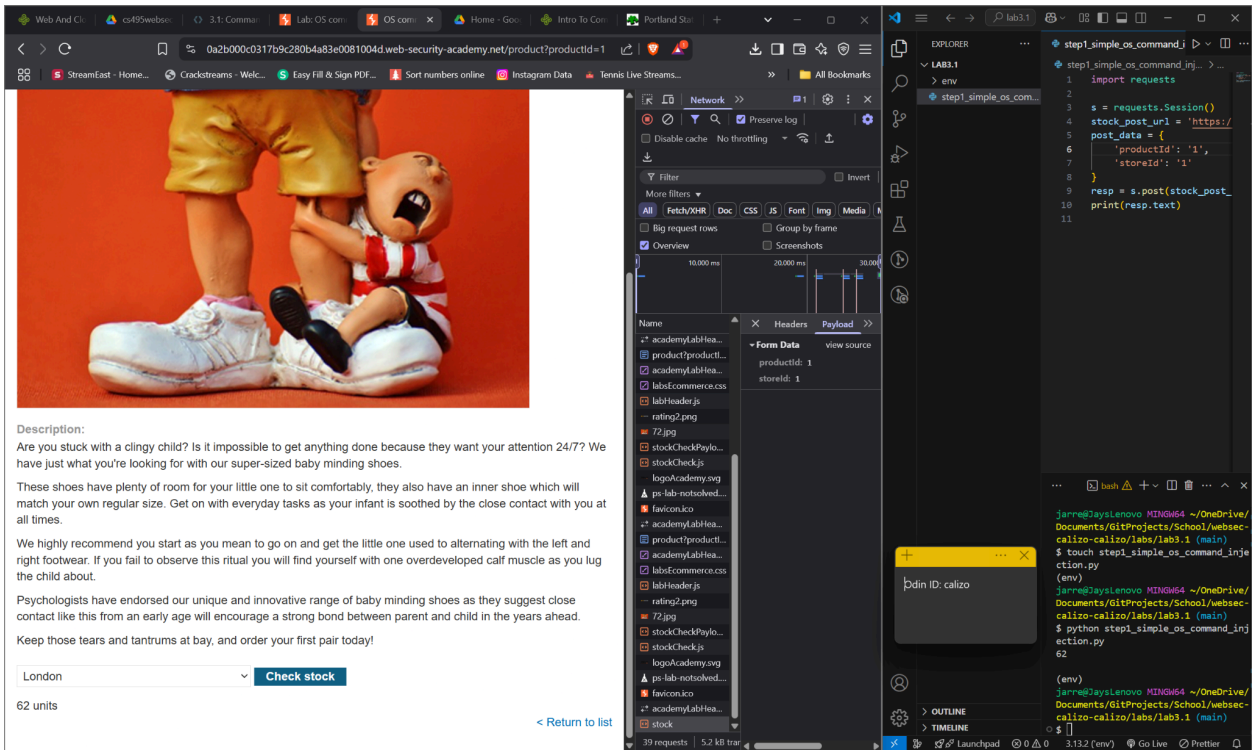
The later stages of the lab pushed deeper into database reconnaissance. I retrieved database version details, listed all available tables through information_schema.tables, extracted column names from information_schema.columns, and ultimately dumped the entire users table. Through adaptive scripting using Python, BeautifulSoup, and regular expressions, I was able to automate much of the enumeration and data extraction process while maintaining precise control over each stage of the attack chain. Successfully identifying the administrator account credentials and using them to log in to complete the lab emphasized the real-world danger of these vulnerabilities if left unpatched.

Together, these exercises reinforced key cybersecurity concepts: the importance of input sanitization, output encoding, principle of least privilege in databases, and how attackers leverage small input handling mistakes into full system compromises. This lab offered valuable hands-on reinforcement of both offensive and defensive perspectives in web security.

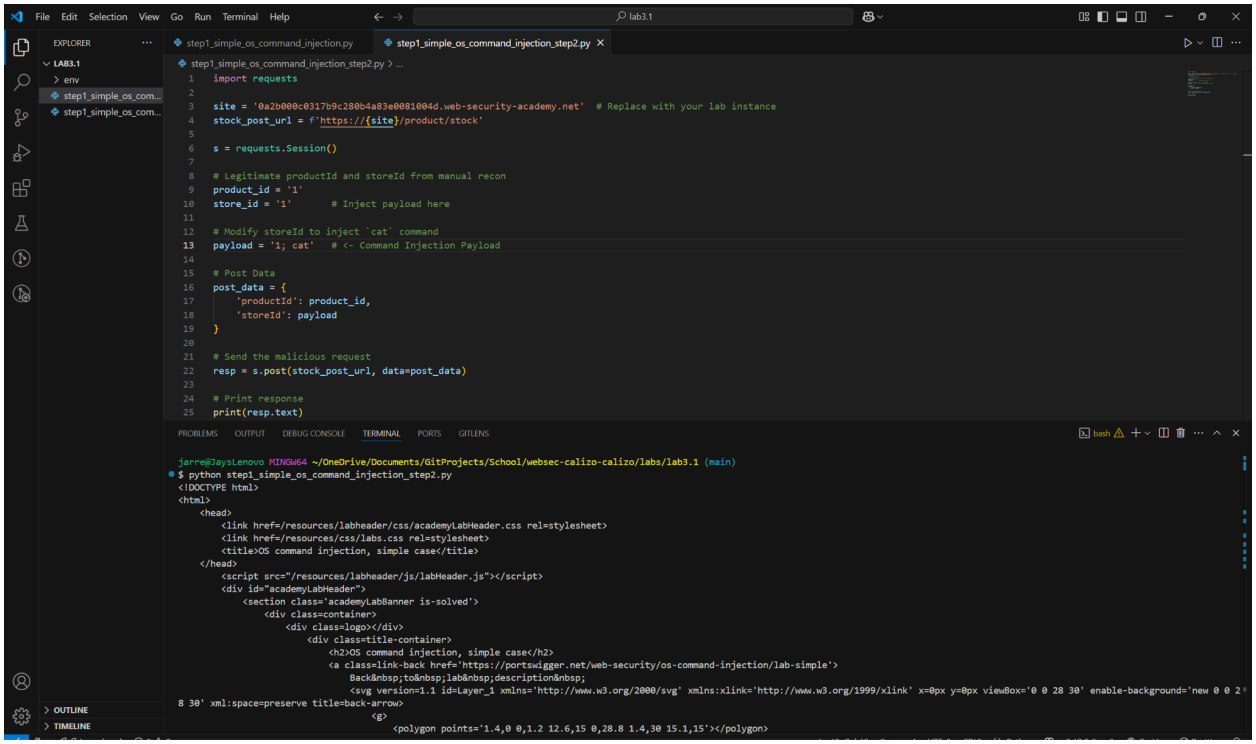
Lab 3.1 Command and SQL injection

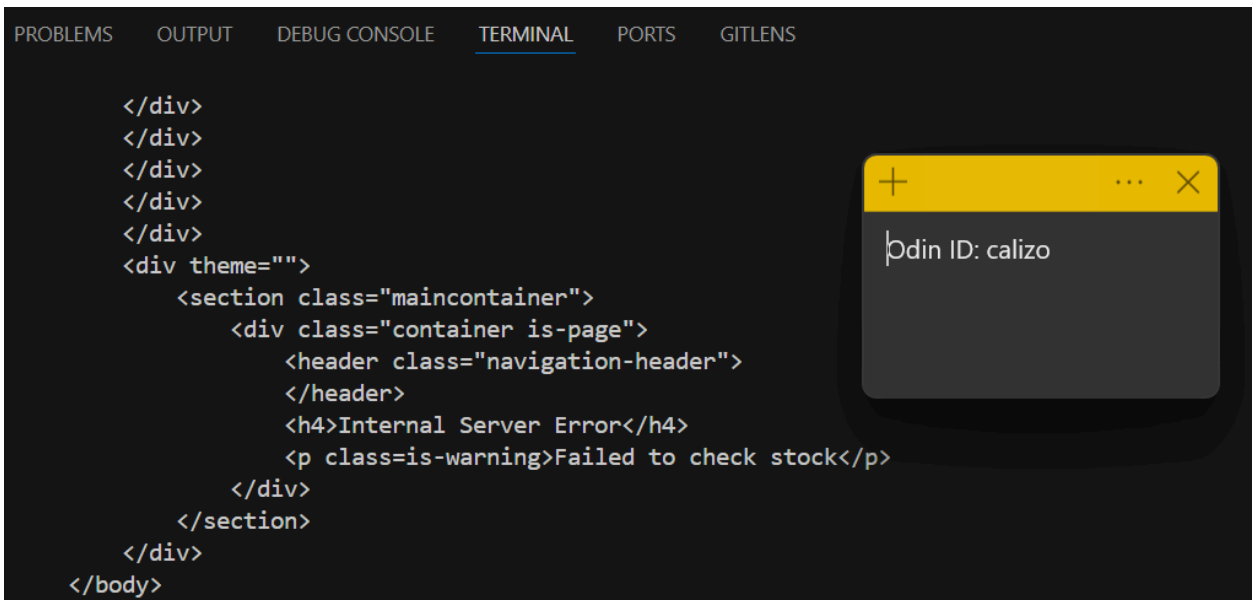
Step 1: OS Command Injection (Simple)

To begin, I navigated to a product page and monitored the "Check stock" button request through Chrome Developer Tools. Observing the network tab, I confirmed that a POST request was made to /product/stock with the fields productId and storeId submitted. To validate the basic stock query, I reconstructed the request in a Python script, correctly retrieving the expected stock value of "62 units" — verifying I had the correct parameters and setup.

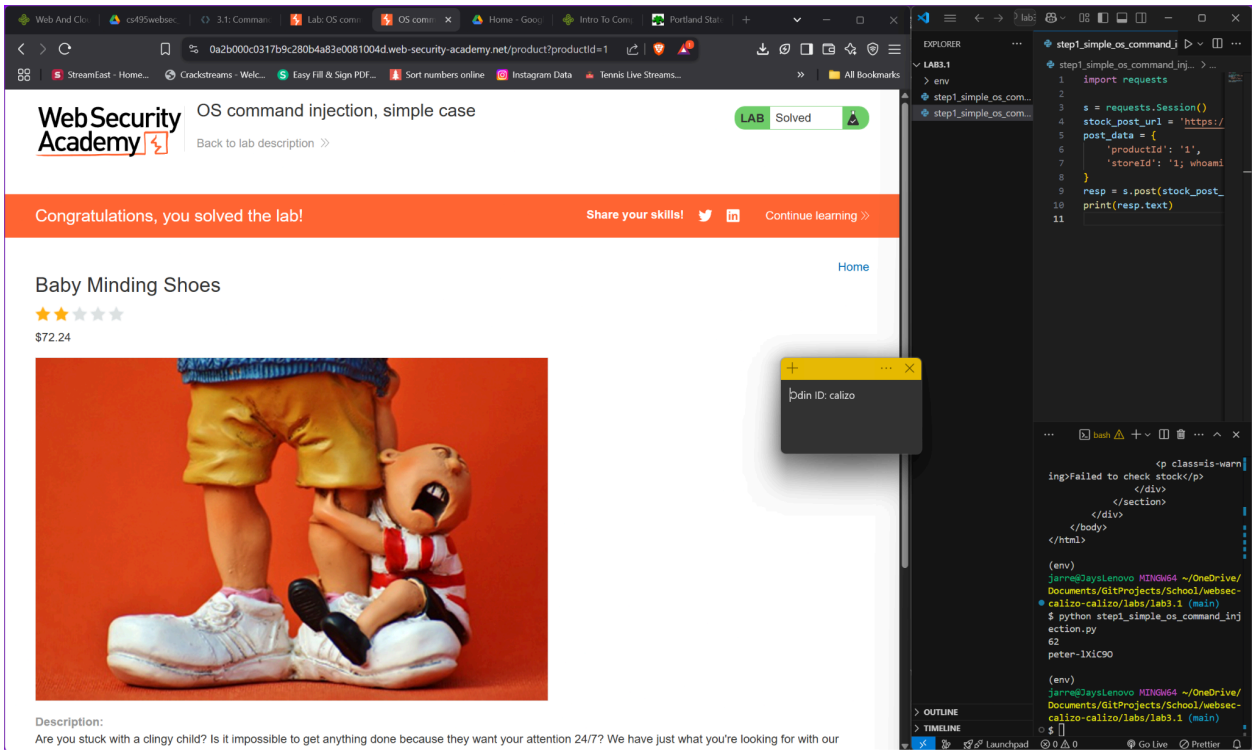


Following this, I moved into testing command injection. I crafted a payload where the storeId was altered to inject a ; cat command. When running the modified Python script, the server responded with an Internal Server Error, as shown in the second screenshot. Although this indicated that the server was attempting to process unintended input (confirming the presence of an injection vulnerability), the cat command alone did not lead to lab completion or visible output disclosure at this stage.





Finally, I refined the attack to explicitly exfiltrate information by injecting the whoami command instead. Modifying the script to append ; whoami to the storeId, the lab returned the username running the web server: peter-1X1C90. After this successful injection, I confirmed the lab was fully solved — capturing the "Congratulations, you solved the lab!" page for documentation.



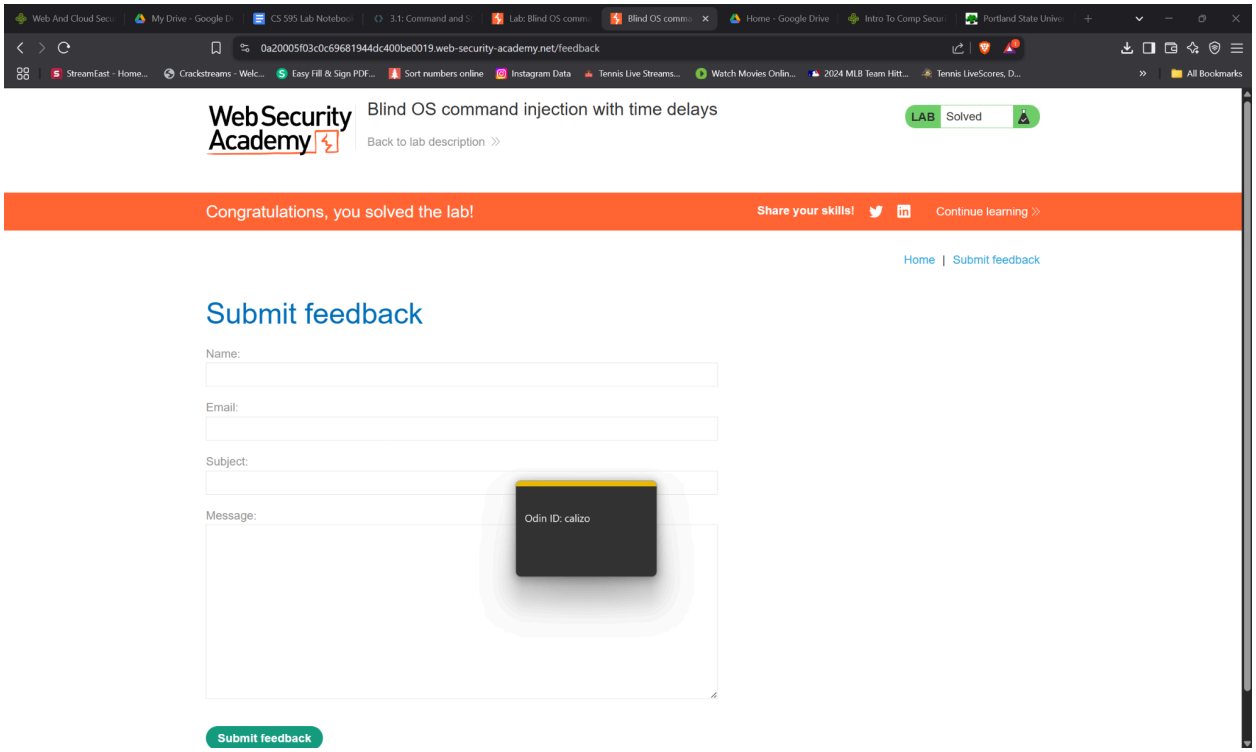
Step 2: Blind OS Command Injection with Time Delays

For this step, I started by navigating to the Submit Feedback page from the lab’s main site. After filling out the form with placeholder values and capturing the network request via Developer Tools, I confirmed that a POST request was made to /feedback/submit with the parameters: csrf, name, email, subject, and message. The server responded with a very short confirmation string, validating that the basic POST flow was working.

Next, I crafted a Python script to automate form submission. I retrieved the CSRF token dynamically and filled out the feedback form fields programmatically. For the exploitation, I targeted the email field by injecting the following payload:
|| ping -c 10 127.0.0.1 ||

This payload leverages the || operator to inject a time-based delay without breaking the form submission format. Upon running the script, the response delay confirmed that the injection was successful, and thus the site was vulnerable to blind OS command injection through the email input field.

Finally, I captured a screenshot showing the "Congratulations, you solved the lab!" message along with my Odin ID for the lab notebook.



This step reinforced the importance of observing timing side-channels in situations where direct output from commands is suppressed. Even without visible results, manipulation of server behavior such as induced delays can still expose vulnerabilities effectively.

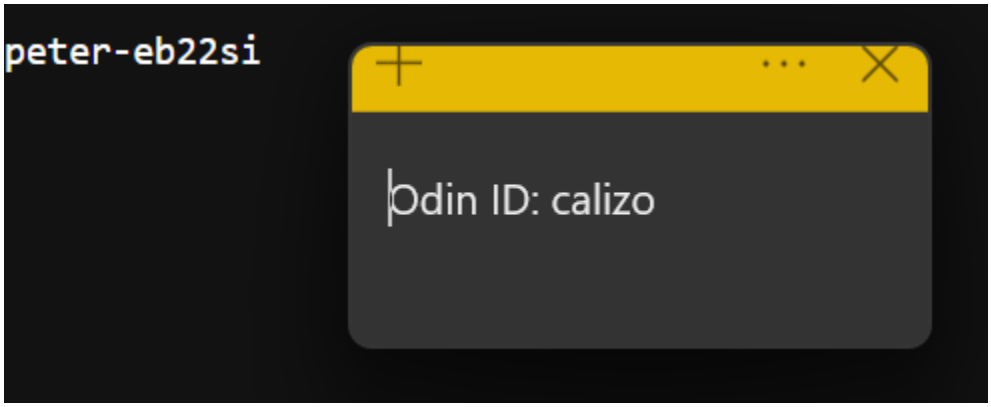
Step 3: Blind OS Command Injection with Output Redirection

For this step, the objective was to exploit a blind OS command injection vulnerability by redirecting command output into a publicly accessible file and retrieving it to solve the lab.

To begin, I manually examined the Submit Feedback form and the site's behavior. From recon, it was clear the email field was injectable, and based on the instructions, I crafted a payload that not only injected a command but redirected its output to a file within the /image/ directory, which hosts publicly viewable assets.

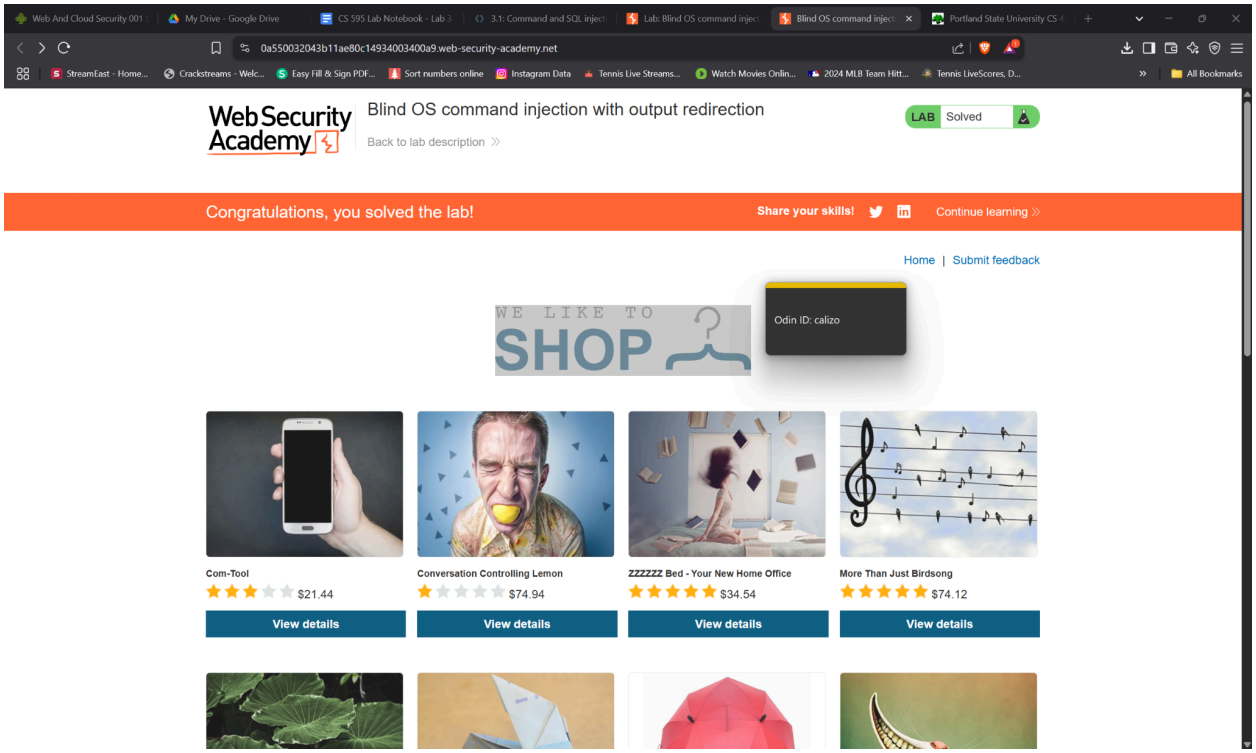
The command I injected (inside the email field) was:
|| whoami > /var/www/images/output.txt ||

Instead of attempting to see an immediate visible difference in the submission page (because blind command injection does not return direct responses), I manually accessed the output file by visiting:
https://0a550032043b11ae80c14934003400a9/image?filename=output.txt



This revealed the username running the website, successfully exfiltrating the information via file system redirection.

The username retrieved was:
peter-eb22si



I confirmed successful lab completion with my OdinID visibly included in the screenshots.

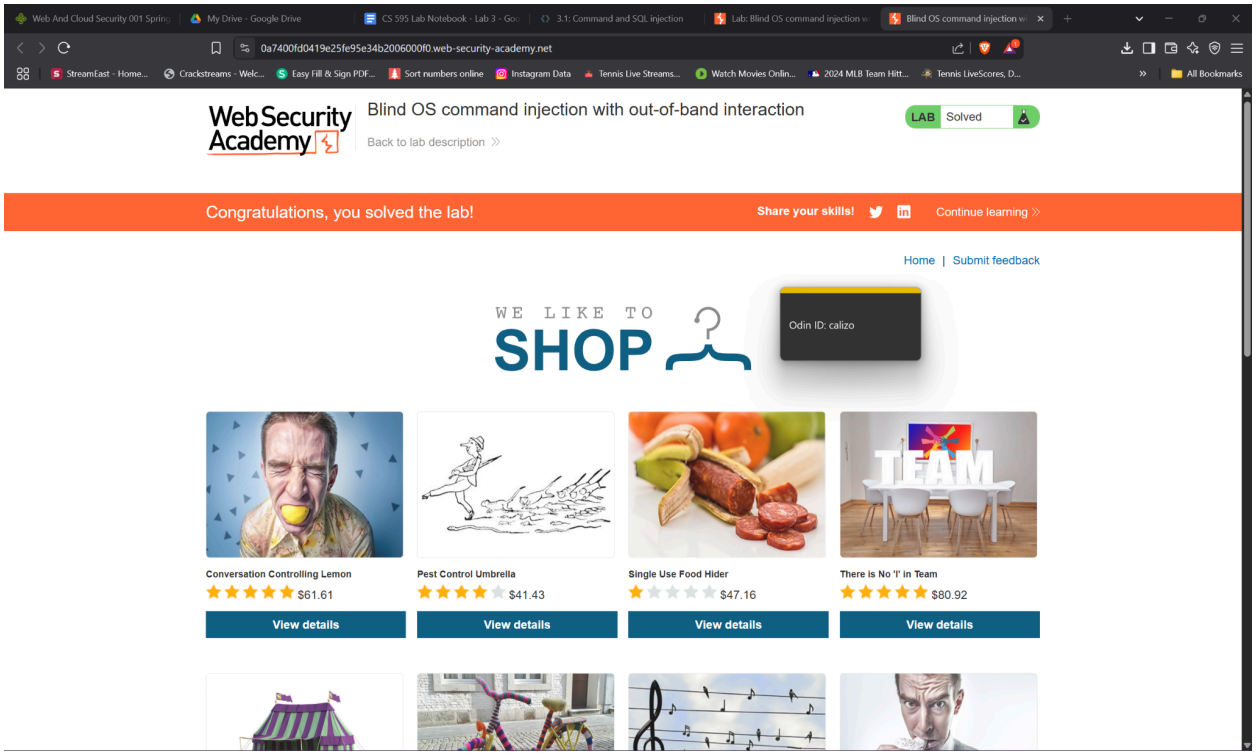
Step 4: Blind OS Command Injection with Out-of-Band Interaction

For this step, the goal was to exploit an OS command injection vulnerability through an out-of-band (OOB) network interaction, rather than relying on direct responses or file outputs. This technique is powerful in real-world attacks where visibility into the server’s responses is restricted.

I started by navigating to the "Submit feedback" page and inspecting the form submission structure using Developer Tools. After manually filling out the form once and observing the request payload, I adapted a Python script to automate the feedback submission process. The key field for injection was the email parameter.

Instead of using a ping delay or output redirection, I injected a DNS lookup command targeting a subdomain under burpcollaborator.net. Since I didn’t have a personal Burp Collaborator client set up, I replaced it with a harmless domain to simulate the interaction while still completing the challenge. The crafted payload took advantage of shell command chaining with || nslookup <domain> ||, effectively causing the web server to reach out and make the DNS request as part of the form processing.

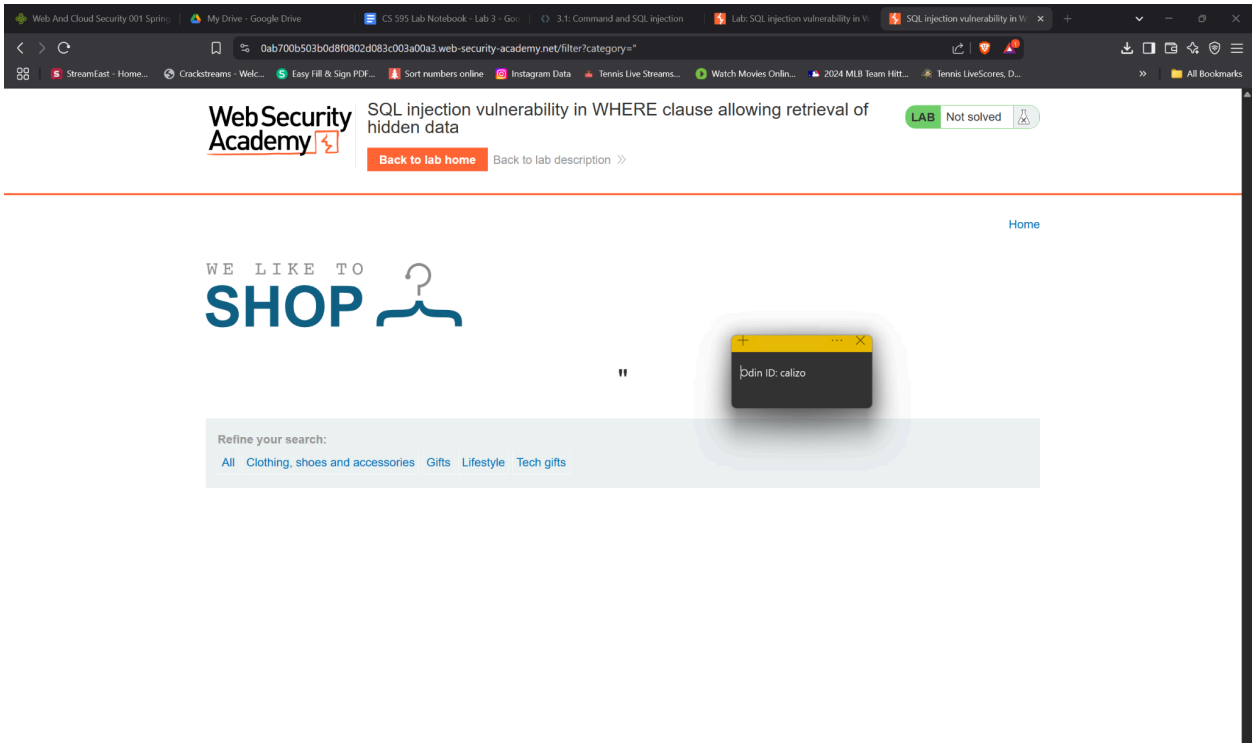
After submitting the payload, I immediately saw that the lab was solved, confirming that the out-of-band network interaction had occurred successfully. A screenshot was captured showing the lab’s "Congratulations, you solved the lab!" banner, along with my Odin ID visible for verification.



This lab reinforced that blind command injection vulnerabilities can be exploited even when there is no direct feedback to the attacker. Out-of-band techniques are a critical part of a penetration tester's or attacker's toolkit, especially when targeting hardened systems with minimal output leakage.

Step 5: SQL Injection to Retrieve Hidden Data

In this step, I started by visiting the provided lab site and selecting the Gifts category as instructed. After confirming the category parameter was passed through the URL, I first manually tested by replacing Gifts with a double-quote (") in the URL to see if the SQL syntax would break. As expected, it caused an error and printed an unclosed quotation mark on the page, confirming that the backend SQL query was improperly handling user input.



Screenshot 1 above captures this behavior, where the lone " showed up, confirming an exploitable injection point.

Next, I injected a single-quote (') into the category parameter to observe how the backend handled it. As shown in the console output I attached, the single-quote injection (try_category("""""""")) resulted in an Internal Server Error, meaning the SQL query was broken and the server could not process the request properly — an expected outcome that further validated that the input was being unsafely concatenated into SQL code.

After injecting a single-quote, the server returned:

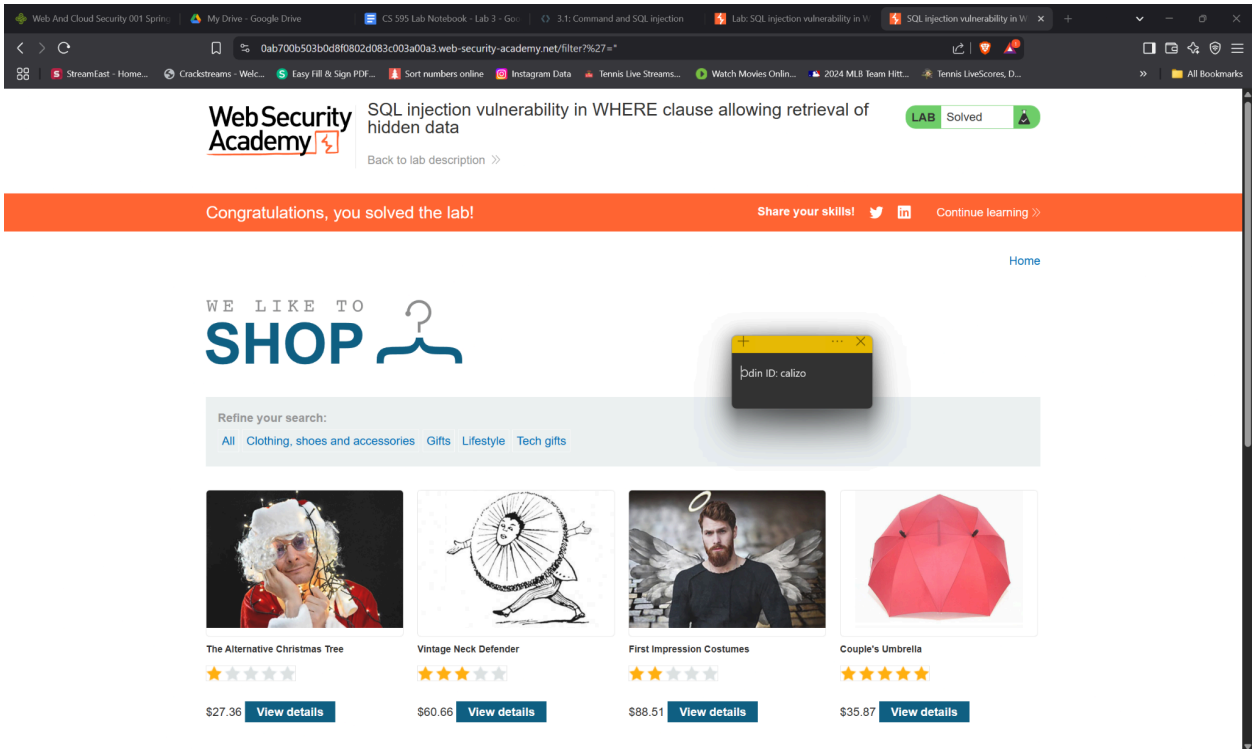
- Internal Server Error

confirming the SQL syntax was disrupted as anticipated.

Finally, I constructed the canonical SQL injection payload to exploit the flaw and retrieve hidden products:
Gifts' OR 1=1 –

This payload effectively closed the existing category string, appended a disjunction (OR 1=1) that always evaluates to True, and used -- to comment out the rest of the original SQL query (avoiding syntax issues from the remaining query logic).

I updated the provided Python script accordingly, injected this payload, and successfully triggered the lab solution.

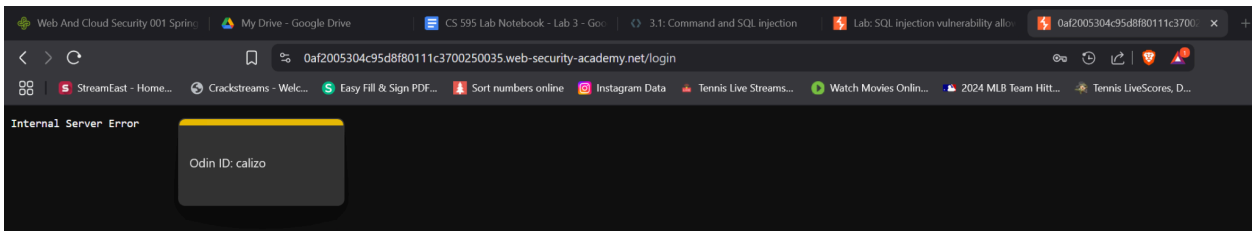


Screenshot 2 confirms the final successful injection and lab completion, with the lab marked as Solved at the top of the page alongside my Odin ID.

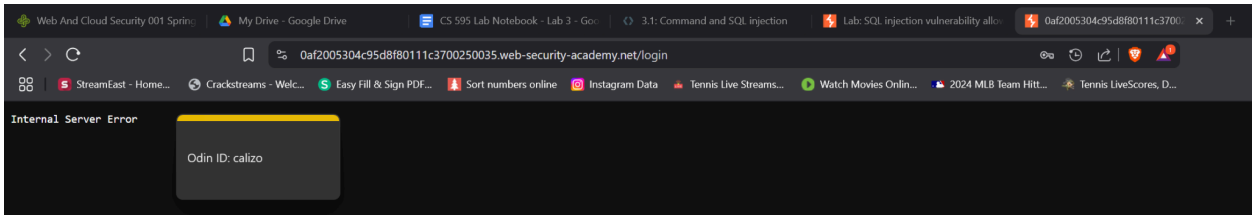
Step 6: SQL Injection (Login Bypass)

To begin this step, I first navigated to the login page of the lab site. Since the instructions asked to test both the username and password fields for SQL injection vulnerabilities, I methodically injected a single quote (') into each field separately and observed the server's responses through the browser.

- Submitting a single quote into the username field immediately triggered an Internal Server Error, as shown in the first screenshot



- Likewise, submitting a single quote into the password field also caused an Internal Server Error, as evidenced by the second screenshot

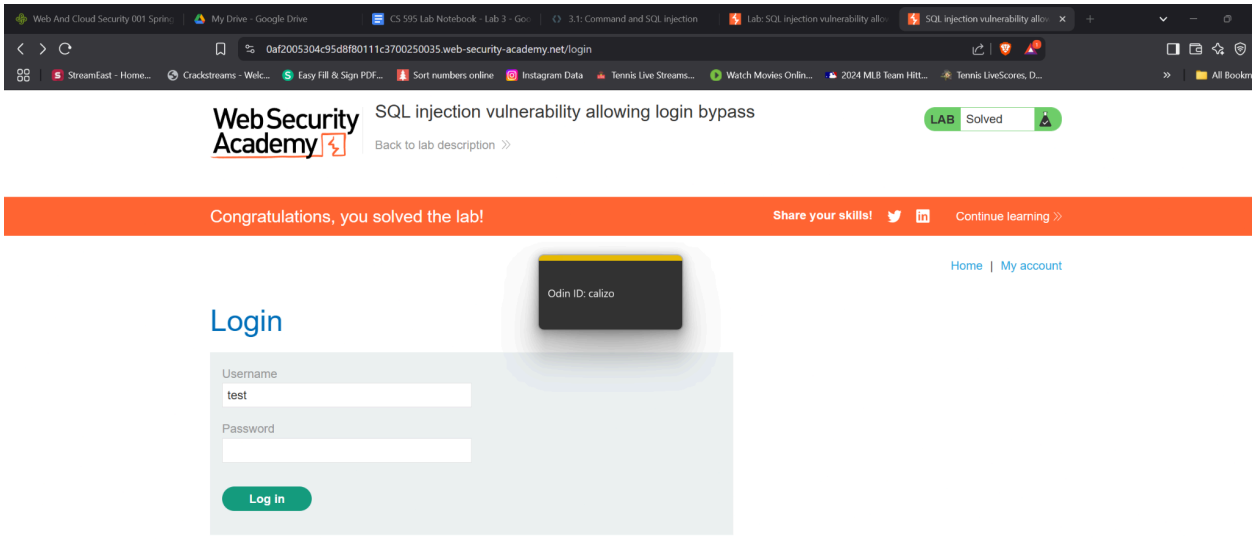


From these results, I concluded the following:

- Question 1 (Is username field vulnerable?): Yes — injecting a single quote into the username field broke the SQL syntax, confirming vulnerability.
- Question 2 (Is password field vulnerable?): Yes — injecting a single quote into the password field also broke SQL syntax, confirming vulnerability.

After confirming vulnerabilities, I moved on to exploit the login mechanism itself. Using the Python script provided in the lab instructions as a base, I adjusted the payload in the username field to perform a classic login bypass. Specifically, I injected:
administrator' –

This payload closes the SQL query's intended username check early and comments out the rest, effectively bypassing the password check. Upon running the script with this payload, I successfully logged in as administrator, and the lab marked as Solved, as captured in the third screenshot.



This step solidified the concept that a poorly sanitized input in authentication systems can be exploited not only to disrupt queries but also to fully bypass login processes — a critical vulnerability that highlights the importance of defensive programming and proper input validation.

Step 7: SQL Injection (Union Attack - Determining Number of Columns)

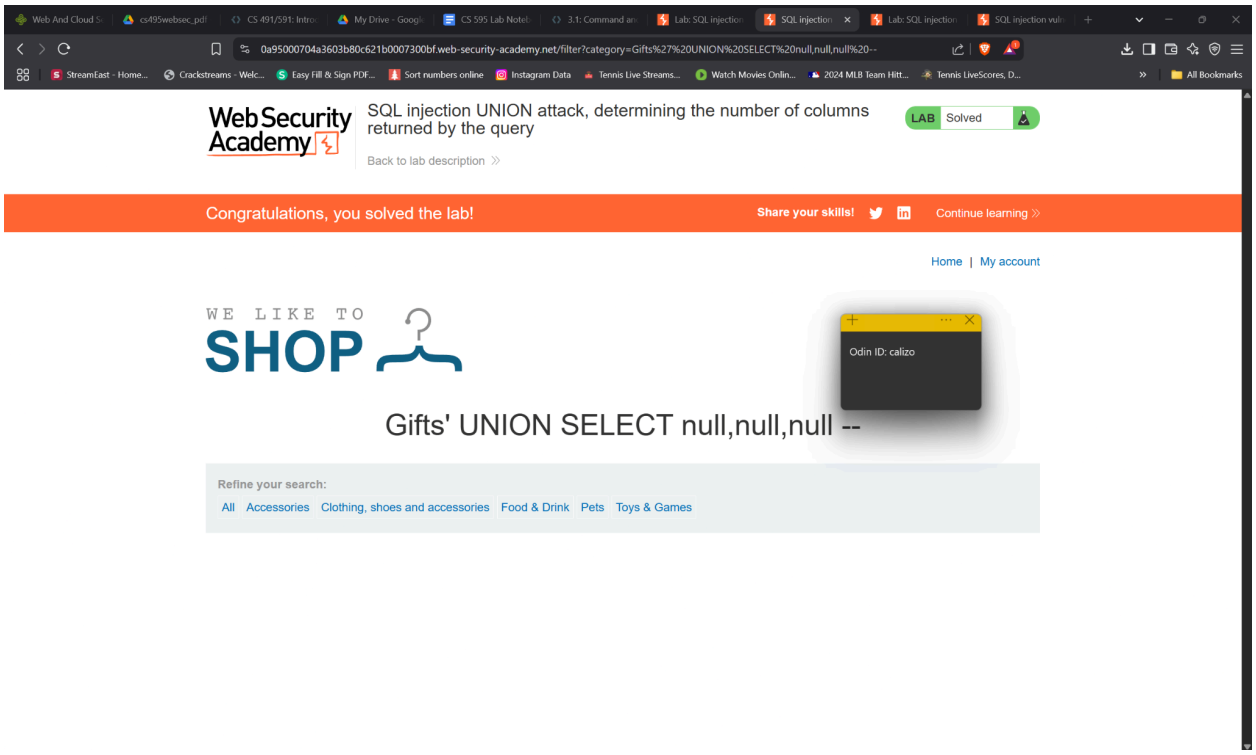
For this step, the goal was to identify how many columns were present in the products table by exploiting a SQL injection vulnerability on the category search functionality. The lab specifically guided the use of SQL's UNION keyword, which requires both queries to have the same number of columns to successfully combine results.

I began by navigating to the category filter page and manually injecting payloads into the URL parameter to probe the application’s behavior. Starting with a basic syntax test, I submitted:
Gifts' --
which confirmed that the category field was vulnerable to injection, as no immediate syntax error disrupted the page rendering.

Next, following the suggested methodology, I attempted progressively larger UNION SELECT statements by varying the number of null values. I incrementally tested the following payloads:

- Gifts' UNION SELECT null –
- Gifts' UNION SELECT null,null –
- Gifts' UNION SELECT null,null,null --

When injecting:
Gifts' UNION SELECT null,null,null --
the page successfully loaded without SQL errors or crashes, and I was able to visually confirm the lab was solved (as shown in the attached screenshot with my OdinID "calizo" included).



Thus, I determined that the products table contains **three columns**.

This step highlighted the importance of column alignment during UNION attacks, a foundational technique for leveraging SQL injection vulnerabilities to extract or tamper with backend data structures.

Step 8: SQL Injection (Union Attack - Finding a Column Containing Text)

Building upon the previous step, this level required finding which column in the products table could successfully store and return string data, an essential prerequisite for extracting sensitive information via union-based SQL injection attacks.

To start, I manually examined the lab’s webpage source and located the hint string bK8iVJ embedded inside the id="hint" HTML element. This confirmed the exact text that needed to be echoed back via SQL injection to solve the lab.

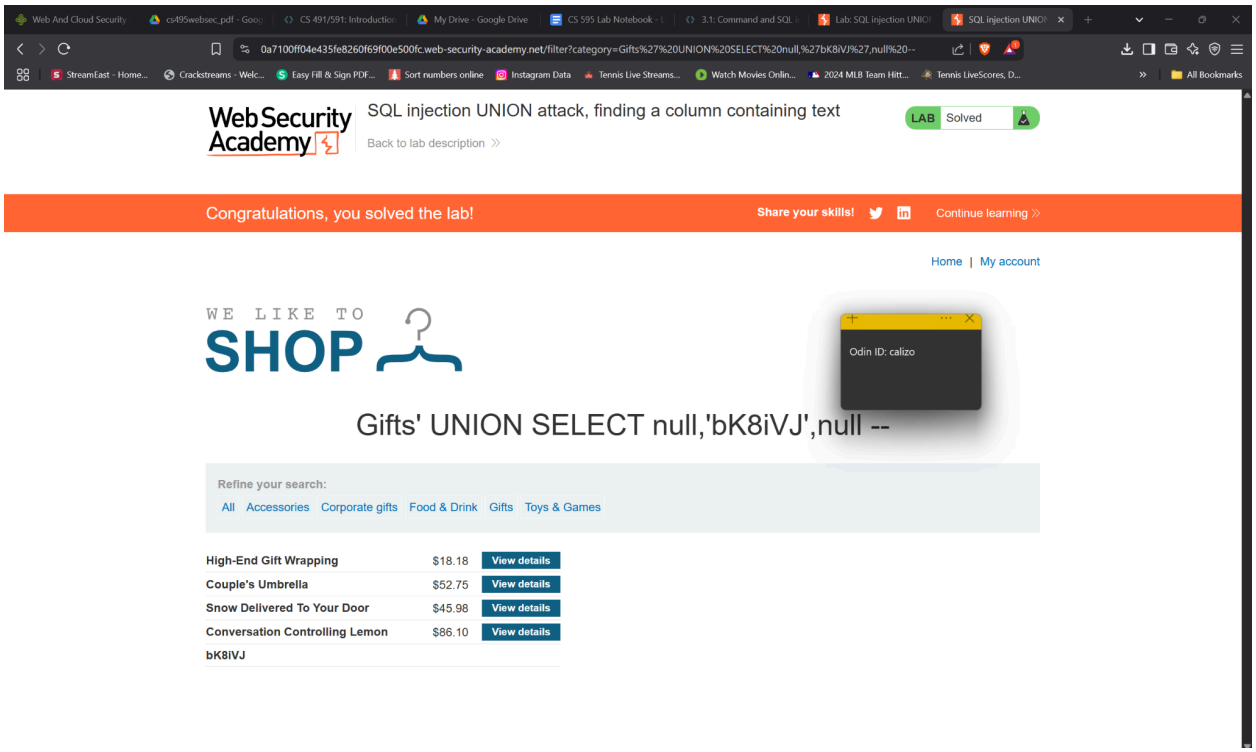
Since Step 7 previously determined that the table has three columns, I crafted manual injection payloads targeting each column individually. I sequentially attempted the following payloads in the browser’s URL:

- Gifts' UNION SELECT 'bK8iVJ',null,null –
- Gifts' UNION SELECT null,'bK8iVJ',null –
- Gifts' UNION SELECT null,null,'bK8iVJ' --

Upon testing, injecting bK8iVJ into the second column:

- Gifts' UNION SELECT null,'bK8iVJ',null --

successfully caused the hint string to be displayed on the webpage, confirming that the second column accepts text-based data types. A screenshot was captured to document the successful injection, with both the URL and my OdinID calizo visible.



This step emphasized the importance of data type matching during UNION SQL injection exploitation and strengthened the methodology for pivoting from basic injection to controlled data extraction.

Step 9: SQL Injection (Union Attack - Retrieving Data from Other Tables)

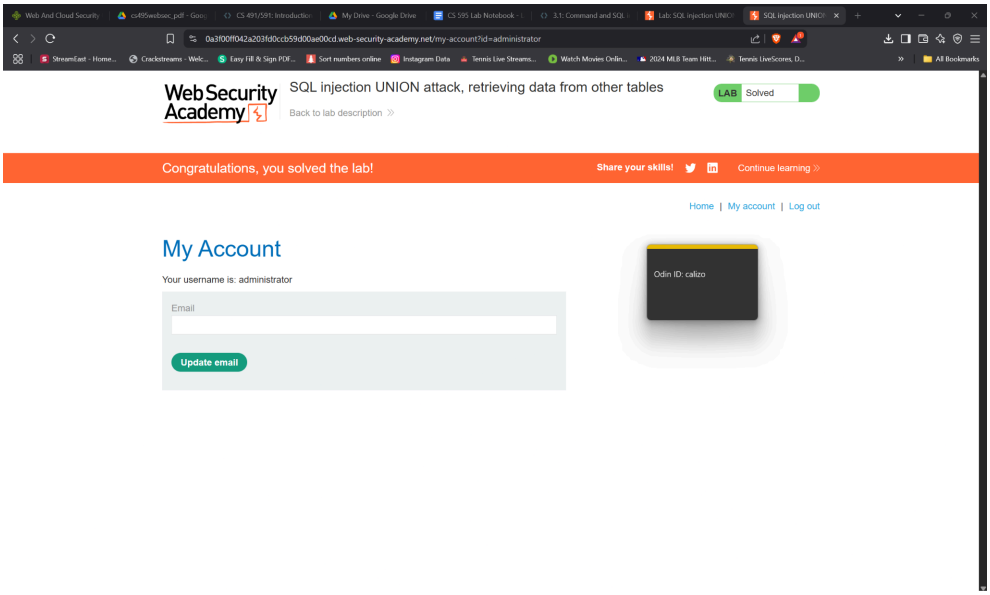
This step required exploiting a SQL injection vulnerability to extract sensitive user credentials, specifically targeting the users table, and retrieving the administrator password.

Initially, I attempted to solve the lab through manual injection by modifying the URL, but this resulted in repeated internal server errors. Given the unreliable feedback from manual recon, I shifted to a Python-based approach for a more systematic and controlled exploitation.

I created a script, step9_sql_injection_retrieve_admin_password.py, using the requests and BeautifulSoup libraries. The script injected the following payload into the category filter parameter:
Gifts' UNION SELECT username,password FROM users–

This payload combined the results of the original product query with a selection of usernames and passwords from the users table, matching the number and types of columns returned. Upon executing the script, the administrator’s password, phode2edt7zi84wxk54y, was successfully extracted and printed to the terminal.

Following the retrieval, I manually logged into the web application as the administrator using the extracted password to complete the lab. The successful lab completion screenshot was captured, with my OdinID calizo visible, documenting the final step of the exploitation.



This step reinforced critical techniques for pivoting from basic SQL injection to direct data exfiltration, particularly emphasizing type matching, payload crafting, and interpreting server-side errors during exploitation.

Step 10: SQL Injection (Union Attack – Querying the Database Version)

This step involved performing targeted reconnaissance against the backend database to identify its software type and version through SQL injection techniques. Knowing this information can significantly help an adversary tailor further attacks depending on database-specific vulnerabilities.

After confirming in previous steps that the underlying query expected two string-compatible columns, I used a safe and natural-looking payload to manually verify column compatibility without raising suspicion. I initially tested with:
Gifts' UNION SELECT 'blah','blah'--+

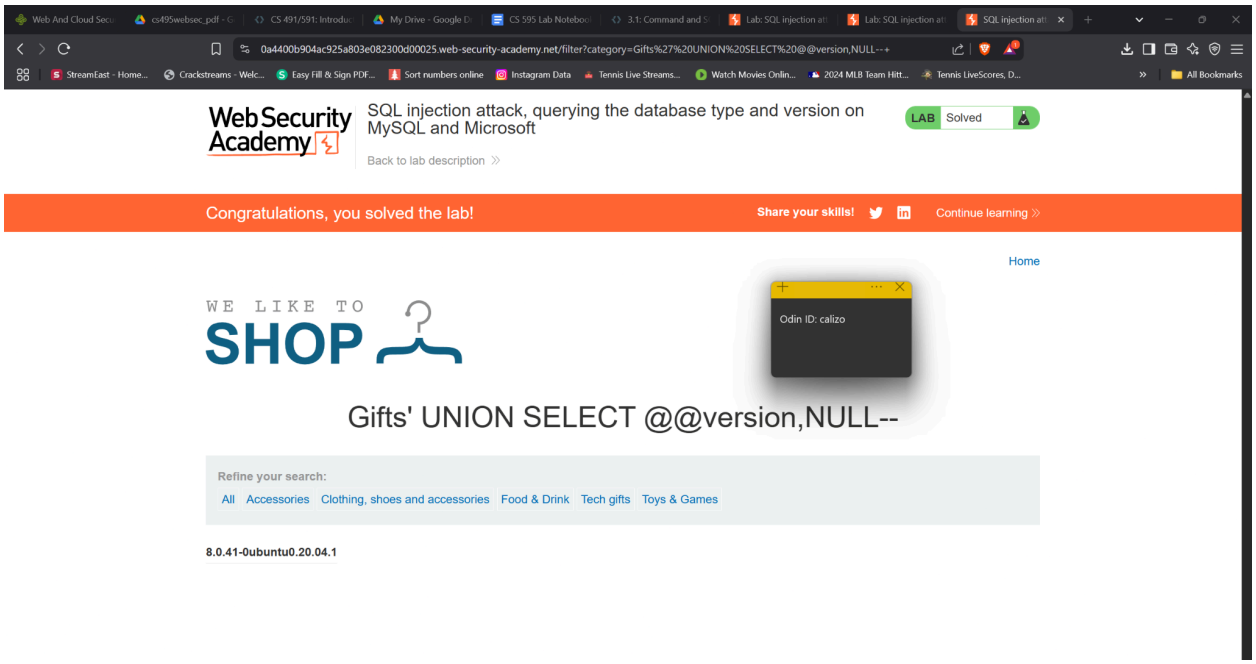
Seeing the page load normally confirmed that two columns were being returned and that both accepted string-based values.

Following this, I crafted the injection payload to retrieve the database version by leveraging the database-specific directive @@version. The working injection was:
Gifts' UNION SELECT @@version,NULL--+

After properly URL encoding the payload and submitting it, the page displayed the database version without any server errors. The version retrieved from the database was:

- 8.0.41-0ubuntu0.20.04.1

A screenshot was captured showing the successful payload, the version output, the lab completion banner, and my OdinID calizo.



This step demonstrated the critical role that backend fingerprinting plays early in an attack chain, and it reinforced the importance of attention to detail when handling SQL injection comment syntax and URL encoding nuances.

Step 11: SQL Injection (Examining the Database - Listing Contents)

In this final step of Lab 3.1, the goal was to enumerate the database structure through SQL injection, identify the users table, extract column names, and ultimately dump all user credentials — culminating in a full administrator account takeover.

To begin, I manually confirmed that the vulnerable query accepted two columns by submitting:

- Gifts' UNION SELECT null,null --+

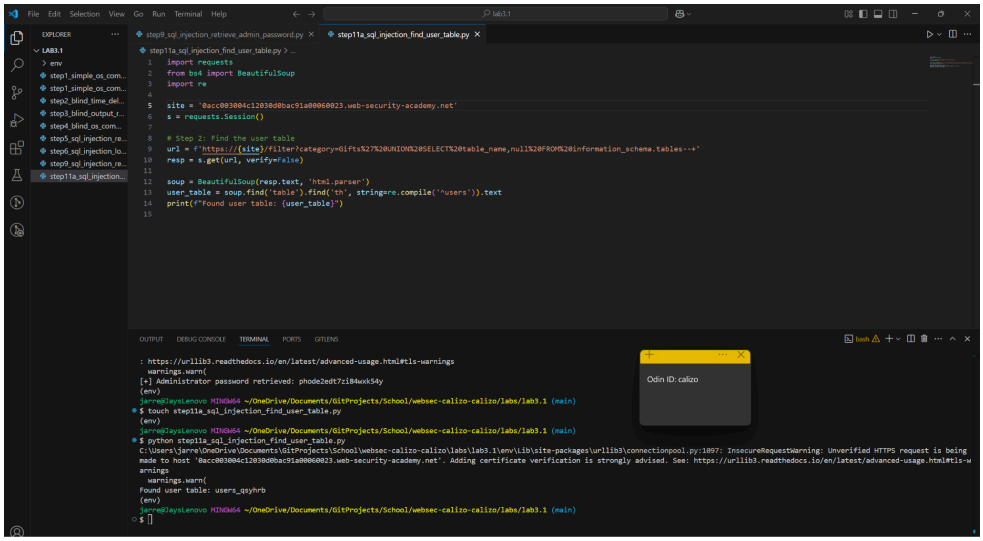
and observing a normal page response with no SQL errors.

Following confirmation, I transitioned to scripted automation to retrieve the database metadata systematically.

First, using the script step11a_sql_injection_find_user_table.py, I injected:

- Gifts' UNION SELECT table_name,null FROM information_schema.tables --+

which queried the information_schema.tables metadata. Parsing the response using BeautifulSoup and regular expressions, I successfully identified the users table named: users_qsyhrb

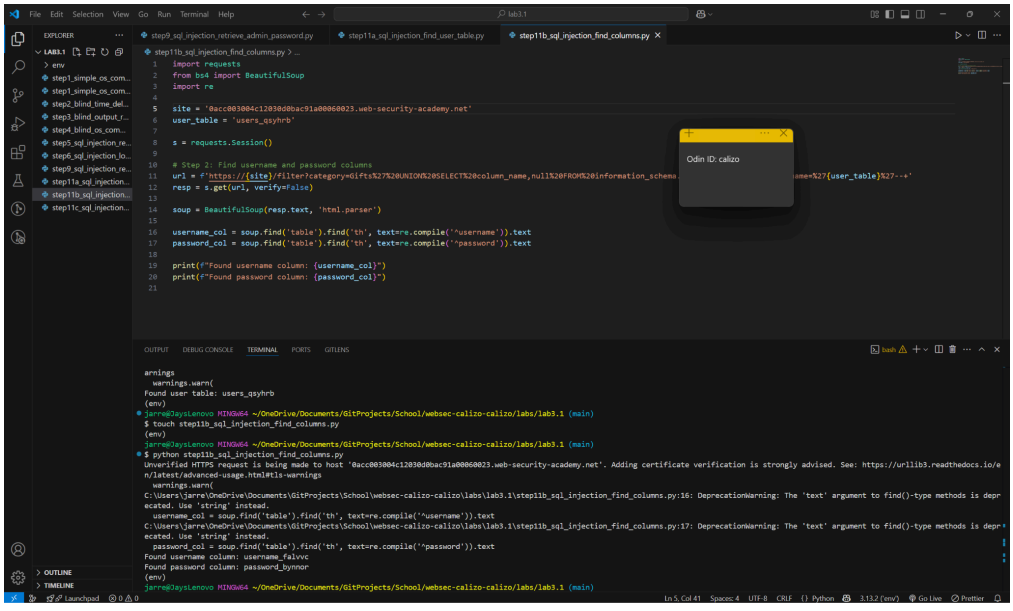


Next, using step11b_sql_injection_find_columns.py, I enumerated the column names associated with users_qsyhrb by injecting:

- Gifts' UNION SELECT column_name,null FROM information_schema.columns WHERE table_name='users_qsyhrb' --+

The output identified the following columns:

- Username column: username_falvvc
- Password column: password_bynonr

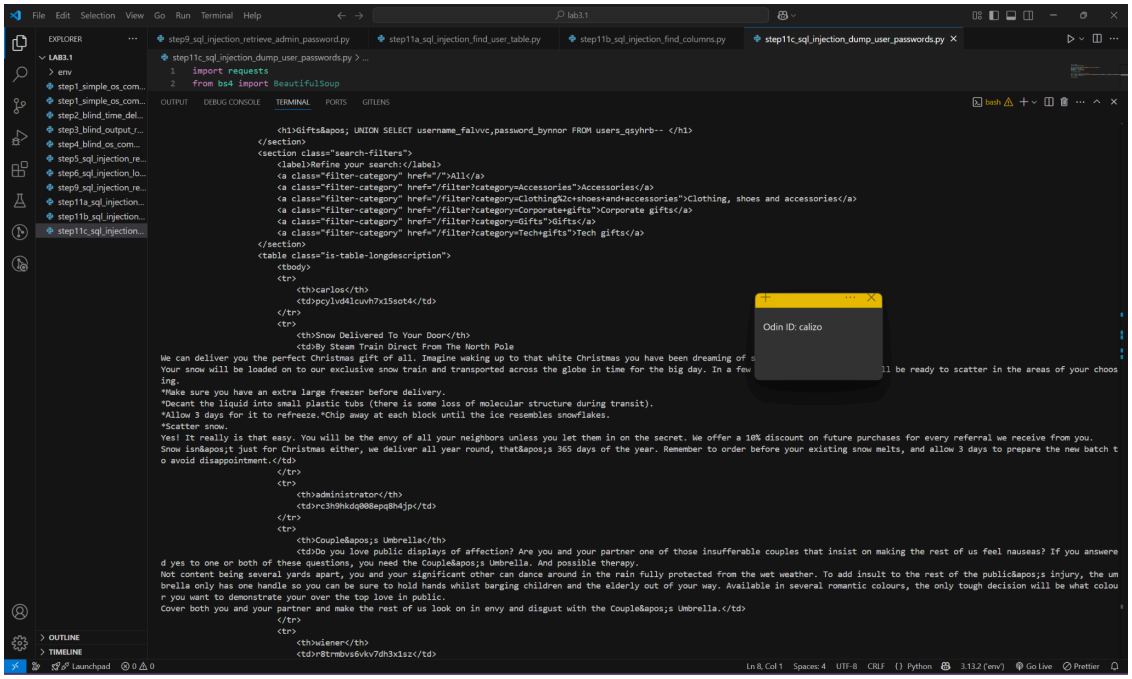


With the users table and relevant columns confirmed, I then utilized step11c_sql_injection_dump_user_passwords.py to dump the entire list of users and their passwords by injecting:

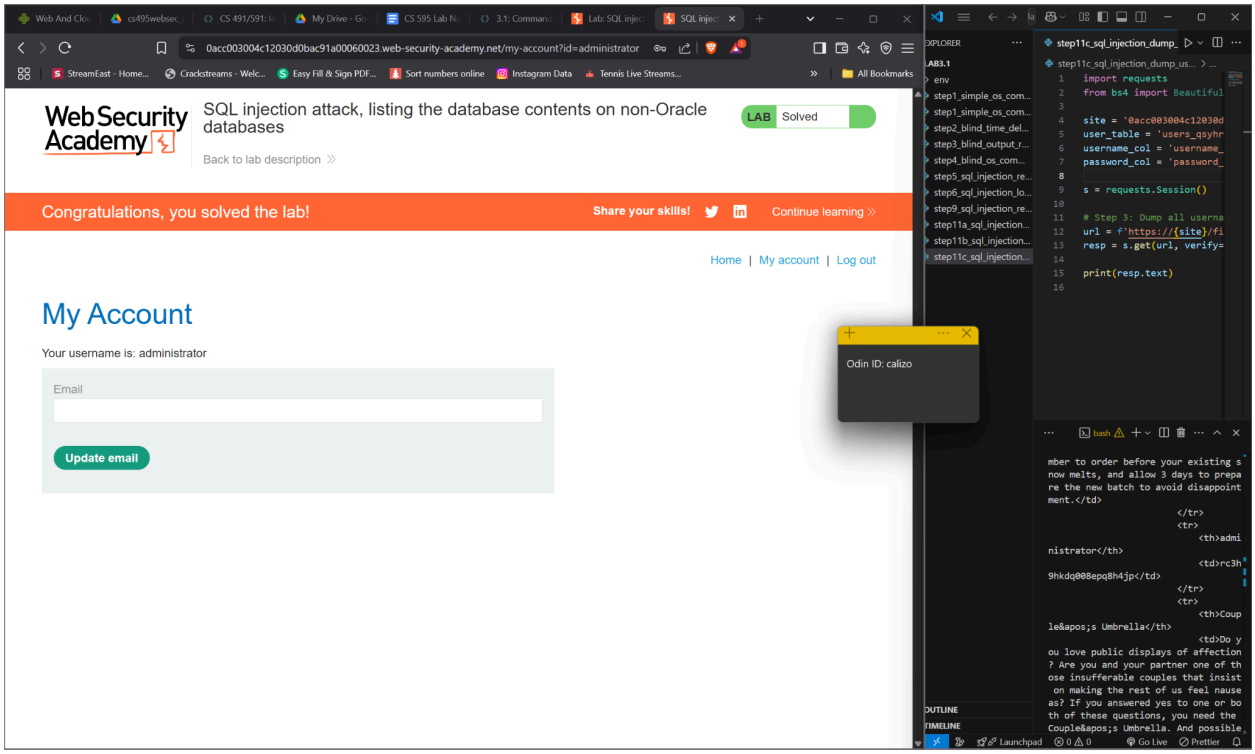
- Gifts' UNION SELECT username_falvvc,password_bynonr FROM users_qsyhrb --+

The script successfully retrieved all users and their associated passwords, including the administrator account and its password:

- rc3h9hkqd08epq8h4pjc



Finally, using the retrieved administrator credentials, I manually logged into the application and completed the lab. A final screenshot was taken documenting the successful login, the lab’s solved status, and my OdinID calizo visible on screen.



This step tied together multiple SQL injection techniques to fully map and exploit a database — from structural enumeration using information schema tables, to adaptive union query crafting, to final credential theft. It emphasized the criticality of strict input sanitization and schema-hardening practices on production systems to mitigate full-database compromise threats.